

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: **Operating Systems**

COURSE CODE: **CSA0404**

COURSE OUTCOME COVERED

C04: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.

Assessment Tool Weightage: 30%

QUESTIONS: 5

Total Marks:25

Annexure A

COMPARATIVE ANALYSIS

Code of Conduct:

*I BP VENKATA UDAY KUMAR 192411203 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

UDAY KUMAR

Signature of the student:

Annexure A

COMPARATIVE ANALYSIS

S.No	Comparative Analysis Questions	Marks
1	Compare Contiguous and Linked file allocation methods in terms of access speed, memory utilization, and fragmentation. Identify which method is more suitable for sequential file access and justify your answer.	5
2	Differentiate between Linked and Indexed file allocation with respect to random access capability, storage overhead, and reliability. Explain which method is more efficient for large files.	5
3	Compare FCFS and SSTF disk scheduling algorithms based on seek time, fairness, and the possibility of starvation. Discuss which algorithm performs better under heavy disk load.	5
4	Analyze the differences between SCAN and C-SCAN disk scheduling algorithms in terms of head movement, response time, and uniformity of service. Which algorithm is more suitable for real-time systems and why?	5
5	Compare LOOK and C-LOOK disk scheduling algorithms with respect to efficiency, total head movement, and practical implementation. Explain which algorithm provides better performance and under what conditions.	5

1. Compare Contiguous and Linked file allocation methods in terms of access speed, memory utilization, and fragmentation. Identify which method is more suitable for sequential file access and justify your answer.

Ans:

Contiguous vs Linked File Allocation

Feature	Contiguous Allocation	Linked Allocation
Access speed	Very fast, especially sequential and random access	Slower because blocks must be followed through pointers
Memory utilization	May waste space due to external fragmentation	Better utilization; blocks can be placed anywhere
Fragmentation	Suffers from external fragmentation	No external fragmentation, but pointer space is required
Random access	Excellent	Poor
Sequential access	Excellent	Good
Overhead	Very low	Extra pointer required in each block

Suitable for sequential file access: Contiguous allocation is generally more suitable because consecutive blocks are stored next to each other. The disk head can read them continuously, resulting in lower seek time and faster access.

2.Differentiate between Linked and Indexed file allocation with respect to random access capability, storage overhead, and reliability. Explain which method is more efficient for large files.

Ans:

Linked vs Indexed File Allocation

Feature	Linked Allocation	Indexed Allocation
Random access	Poor; blocks must be traversed sequentially	Excellent; index block directly points to required blocks
Storage overhead	Pointer is stored in every data block	Requires one or more index blocks
Reliability	Lower; a damaged pointer can make the remaining blocks inaccessible	Better; index maintains addresses of all blocks
Sequential access	Good	Good
Large files	Can become slow because of pointer traversal	More efficient for accessing large files

More efficient for large files: Indexed allocation is generally more efficient because it supports direct/random access and avoids traversing a long chain of pointers. For very large files, multiple-level indexing may be used.

3. Compare FCFS and SSTF disk scheduling algorithms based on seek time, fairness and the possibility of starvation. Discuss which algorithm performs better under heavy disk load.

Ans:

FCFS vs SSTF Disk Scheduling

Feature	FCFS	SSTF
Meaning	First Come First Serve	Shortest Seek Time First
Seek time	Usually high	Usually lower
Fairness	Very fair; requests are served in arrival order	Less fair
Starvation	No starvation	Starvation is possible
Implementation	Very simple	More complex
Performance	Lower overall performance	Better average seek performance

Under heavy disk load

SSTF generally performs better because it selects the request closest to the current head position, reducing average seek time.

However, SSTF can cause starvation when requests far from the current head position continuously wait while nearby requests are repeatedly served.

Conclusion:

- FCFS: Better fairness and simplicity.
- SSTF: Better average seek time and performance under heavy load.

4. Analyze the differences between SCAN and C-SCAN disk scheduling algorithms in terms of head movement, response time, and uniformity of service. Which algorithm is more suitable for real-time systems and why?

Ans:

SCAN vs C-SCAN Disk Scheduling

Feature	SCAN	C-SCAN
Head movement	Moves in both directions, like an elevator	Serves requests in only one direction
Response time	Good, but can vary depending on position	More predictable
Service uniformity	Less uniform	More uniform
Efficiency	Good	Slightly lower due to return movement
Fairness	Good	Very good
Starvation	Very unlikely	Very unlikely

Which is better for real-time systems?

C-SCAN is generally more suitable when predictable and uniform response time is important.

The disk head serves requests in one direction. After reaching the end, it returns to the beginning without servicing requests during the return. This gives requests a more uniform waiting time. Conclusion:

- SCAN: Better overall efficiency.
- C-SCAN: Better uniformity and predictability, making it preferable for systems requiring consistent response times.

5. Compare LOOK and C-LOOK disk scheduling algorithms with respect to efficiency, total head movement, and practical implementation. Explain which algorithm provides better performance and under what conditions.

Ans:

LOOK vs C-LOOK Disk Scheduling

Feature	LOOK	C-LOOK
Head movement	Moves in both directions	Moves in only one direction
Total head movement	Usually higher than C-LOOK	Usually lower
Efficiency	Efficient because it doesn't go to disk ends unnecessarily	More efficient for uniformly distributed requests
Implementation	Relatively simple	Slightly more complex
Service pattern	Serves requests in both directions	Serves requests in one direction and jumps back
Response time	Good	More uniform

Which provides better performance?

C-LOOK can provide better performance when:

- There are many disk requests.
- Requests are distributed across the disk.
- Uniform response time is important.
- Reducing unnecessary head movement is a priority.

LOOK can be better when:

- The request pattern is not uniform.
- Simpler implementation is preferred.

- Requests are concentrated in a particular area of the disk.

Final conclusion

LOOK is an improvement over SCAN because the head reverses direction at the last pending request, rather than going all the way to the disk's physical end.

C-LOOK is an improvement over C-SCAN because it also avoids travelling to the physical ends of the disk. Therefore, C-LOOK generally provides lower unnecessary head movement and more uniform service, especially under heavy and uniformly distributed workloads.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

